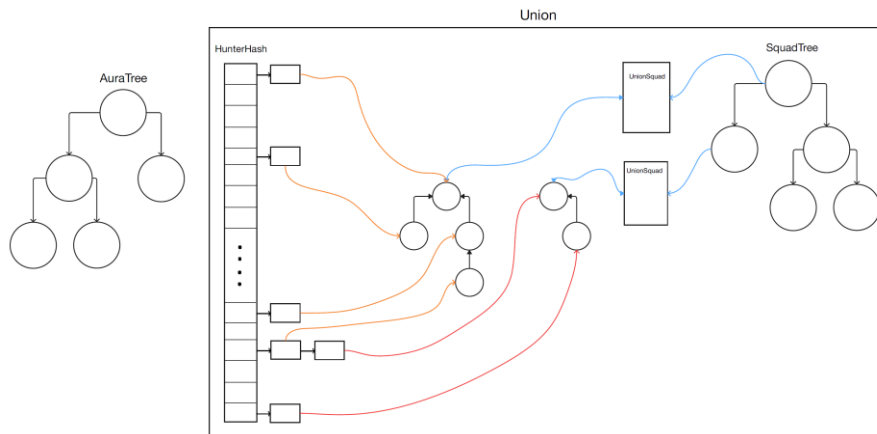


מבני נתונים תרגיל רטוב 2 – חלק יבש:



המערכת שלנו תכיל את המבני הנתונים הבאים:

- **AuraTree** – עץ דרגות המבוסס AVL המחזיק זוגות של $(squadId, Aura)$. הזוגות ממיינים לפי $Aura$ ולאחר מכן ID (במצב של שוויון). תכונת העץ תהיה כמות הצמתים בתת העץ שלו.
- **squads** – מבנה $Union$ שייצג את הקבוצות. מימוש $Union$ בהמשך.

בנוסף נגדיר את העצם הבא:

- $hunter$ יכיל את השדות הבאים:
 - $hunterId$ – מזהה הצייד.
 - $fightHad$ – משתנה קבוע השומר את כמות הקרבות בהן הצייד השתתף ברגע היצירה שלו.
 - $joinedGroupFights$ – משתנה קבוע השומר את כמות הקרבות שהקבוצה השתתפה בהם עד רגע יצירתו.
 - $Aura$ – משתנה המכיל את ההילה שלו.
 - $NenAbility$ – יכולת הנן של הצייד.

במימוש שלנו נשתמש במבנים הבאים:

- **עץ AVL דרגות** –
עץ ה- AVL מהתרגיל רטוב 1 עם הוספת השדה $childrenCount$. שדה זה שומר את מספר הצמתים בתת העץ של כל צומת. נבחין כי לכל צומת, בהינתן ערך זה של שני בניו, חישוב השדה מתבצע ב- $O(1)$. לכן תחזוק השדה לא משנה את סיבוכיות הפעולות של העץ. זאת מכיוון שבמהלך הפעולות שדורשות את עדכנו (הכנסות, גלגולים וכו') נוסף לעדכון שדות הצומת עוד $O(1)$ פעולות.

• **-Hash**

- טבלת ערבול דינמית מבוססת $Hashing$ – $chain$.
נגדיר פרמטר MAX_LOAD_FACTOR . אצלנו, פרמטר זה מוגדר להיות 1.
נגדיר פרמטר $GROWING_FACTOR$. אצלנו, פרמטר זה מוגדר להיות 2.
בנוסף המבנה יכיל את השדות הבאים:
 - $array$ – מערך של $HashNode$.
 - $size$ – גודל המערך. מאותחל להיות 8.
 - $itemCount$ – כמות האיברים במבנה.

פונקציית הערבול תהיה $id \% size$.
נבחין כי פונקציית המודולו ענה על הנחת הפיזור האחד.
כעת, בכל הכנסה, נבדוק אם פקטור העומס עולה על הסף שהגדרנו.
אם כן, ניצור מערך חדש בגודל כפול, נעביר אליו את האיברים (באמצעות rehash) ונמחוק את המערך הישן.
מתרגול, סיבוכיות הזמן המשוערכת של $insert$ בממוצע על הקלט הינה $O(1)$.
בנוסף, מכיוון שאנו דואגים לכך שפקטור העומס יהיה לכל היותר 1 בכל רגע נתון, מתקיים כי תוחלת כל שרשרת במבנה היא מגודל 1.
לכן, בממוצע על הקלט, פעולת החיפוש גם היא מתבצעת ב- $O(1)$.

מבנה ה-Union:

המבנה ממומש באמצעות עצים הפוכים עם שיפור כיוון מסלולים ואיחוד לפי גודל בדומה למבנה ה-Union הנלמד בהרצאה.
לצורך המימוש נגדיר את העצמים הבאים:

- **UnionSquad** - רשומת הקבוצה. אפקטיבית זהו ה- *squad*.
נשמור בה את השדות הבאים:
 - *quadId* – מזהה הקבוצה.
 - *size* – מספר הציידים בקבוצה.
 - *squadNen* – יכולת הנן הכוללת של כל הקבוצה.
 - *squadExp* – ניסיון הקבוצה.
 - *squadAura* – סכום ההילות של חברי הקבוצה.
 - *groupRoot* – מצביע לשורש העץ ההפוך של הקבוצה.
- **UnionNode** - צומת בעץ ההפוך.
נשמור בה את השדות הבאים:
 - *hunter* – הצייד שהמוחזק בצומת.
 - *relFights* – שדה שיאפשר לנו לחשב את כמות הקרבות בהם הצייד השתתף באמצעות סכימת השדה במסלול עד השורש.
 - *relNen* – שדה שיאפשר לנו לחשב את סכום יכולות הנן של חברי הקבוצה שהצטרפו לפני הצייד הנ"ל באמצעות סכימת השדה במסלול עד השורש.
 - *parent* – מצביע להורה של ה- *Node* הנ"ל.
 - *squad* – מצביע לרשומה (*UnionSquad*). מצביע זה מאותחל להיות *nullptr* עבור צמתים שהם לא שורש העץ.

בנוסף, המבנה Union מחזיק את המבנים הבאים:

- *groups* – עץ AVL המחזיק את רשומות הקבוצה *UnionSquad* הממוין לפי *Id*.
זוהי תחלופה למערך הרשומות במימוש הרגיל הנותנת לנו גישה ב- $O(\log k)$ לקבוצה.
- *HunterHash* - מבנה *Hash* דינאמי הממפה בין *HunterId* לבין מצביע ל- *UnionNode*.
זוהי תחלופה למערך צמתים במימוש הרגיל.

כעת נסביר את אופן חישוב ערכי *fights* ו- *partial_nen* של צייד.

-fights

תחילה, אופן חישוב ערך זה עבור כל צייד יהיה:

$$fights = (hunter.fightsHad - hunter.joinedGroupFights) + \sum Node.relFights$$

כאשר הסכימה של $relFights$ מתבצעת על גבי כל הצמתים ממנו עד לשורש העץ (כולל עצמו והשורש).
נבחין כי כאשר נרצה להוסיף לקבוצה קרב, נוסיף ל- $relFights$ של השורש 1.

• **-partial nen**

תחילה, אופן חישוב ערך זה עבור כל צייד יהיה:

$$partial_nen = hunter.NenAbility + \sum Node.relNen$$

כאשר הסכימה של $relNen$ מתבצעת על גבי כל הצמתים ממנו עד לשורש העץ (כולל עצמו והשורש).

נבצע את פעולות מבנה ה- $Union$ באופן הבא תוך כדי תחזוקת השדות $relFights$ ו- $relNen$:

הכנסת צייד חדש - $InsetHunterToGroup$

פונקציה זו מקבלת מזהה קבוצה ואת פרמטרים היצירה של $Hunter$ ומכניסה את הצייד לקבוצה.
אם הצליחה מחזירה 1 אם לא 0 ומפעפעת חריגות זיכרון.
תחילה נחפש את הקבוצה בעץ. אם לא מצאנו נחזיר 0.
לאחר מכן נחפש את הצייד ב- $Hash$. אם הוא קיים כבר נחזיר 0.

אחרת נכניס את הצייד החדש.

ניצור $UnionNode$ עם צייד בעל הפרמטרים המתאימים.

בפרט לצייד ול- $UnionNode$ שלו נאתחל את השדות הבאים:

- $hunter.fightsHad$ להיות הערך המסופק של מספר הקרבות בהם היה הצייד לפני ההצטרפות לקבוצה.
- $hunter.joinedGroupFights$ להיות ערך ה- $relFights$ של שורש העץ.
- $relFights$ להיות 0.
- $relNen$ להיות

$$renNen = squadNen - root.relNen$$

כאשר $squadNen$ היא יכולת Nen הכוללת של כל הקבוצה לפני הצטרפות הצייד.

נעת נעדכן את השדות המתאימים ברשומת הקבוצה $UnionSquad$.

כלומר נעדכן את גודלה, סכום יכולת הנן וסכום ההילות שלה.

נכניס מצביע ל- $UnionNode$ החדש עם ה- $HunterId$ הנ"ל למבנה ה- $Hash$.
לבסוף נחזיר 1.

סיבוכיות הזמן:

נבחין כי חיפוש הקבוצה מתבצע ב- $O(\log k)$ פעולות כאשר k הוא מספר הצוותים במבנה.
כפי שהסברנו במבנה ה- $Hash$, בממוצע על הקלט חיפוש ב- $Hash$ מתבצע ב- $O(1)$.
לאחר מכן אנו מבצעים מספר קבוע של חישובים והשמות של שדות ולכן פעולות עלו מתבצעות ב- $O(1)$.
ולבסוף אנו מכניסים איבר חדש ל- $Hash$ ב- $O(1)$ משוערך בממוצע על הקלט.
כפי שהסברנו סיבוכיות הזמן המשוערכת של פעולות $inert$ היא $O(1)$ בממוצע על הקלט.

לכן הסיבוכיות המשוערכת של הפעולה בממוצע על הקלט היא:

$$O(\log k) + O(1) + O(1) + O(1) = O(\log k)$$

איחוד לפי גודל - Unite :

פונקציה זו מקבלת מזחים של שתי קבוצות A ו- B ומאחדת את הצייד לקבוצה. אם הצלחנו נחזיר 1, אחרת 0.
נניח כי אנו מתבוננים על שתי הקבוצות A ו- B ואנו רוצים לאחד את B ל- A .

כלומר מבחינת סדר הציידים בקבוצות יתקיים

$$a_1 a_2 \dots a_n b_1 b_2 \dots b_m$$

תחילה נחפש את שתי הקבוצות בעץ הרשומות. אם אחת מהן לא קיימת נחזיר 0.
אחרת נתחיל באיחוד:

• $|A| \geq |B|$:

- נצביע את ההורה של שורש B להיות שורש A .
- ערכי A לא צריכים להשתנות ומסלול החישוב שלהם נותר זהה ולכן לא נשנה את $relFights$ ו- $relNen$ של שורש A .
- ערכי $fight$ של B לא צריכים להשתנות אבל מסלול החישוב שלהם עובר כעת דרך שורש A . לכן נתקן את זה בכך שנוריד את $relFights$ של שורש A מ- $relFights$ של שורש B .
כלומר $Broot.relFights = Aroot.relFights$.
- לערכי $partialNen$ של B צריכים להתווסף כל ערכי ה- n של A מכיוון שכעת הם הצטרפו לקבוצה כרונולוגית אחרי חברי קבוצת A . כלומר נרצה להוסיף למסלול הסכימה של ציידים מ- B את יכולת ה- Nen הכוללת של הקבוצה A .
לכן, נוסיף לשדה $relNen$ של שורש B את $squadNen$ של הקבוצה A ונחסיר ממנו את $relNen$ של שורש A .
כלומר נבצע:

$$Broot.relNen += A.squadNen - Aroot.relNen$$

- נעדכן את ערכי רשומת A לייצג את ערכי הקבוצה המאוחדת.
- נמחק את המצביע לרשומה בשורש B ונוציא מעץ הרשומות את רשומת הקבוצה B .

• $|A| \leq |B|$:

- נצביע את ההורה של שורש A להיות שורש B .
- ערכי $fight$ של B לא צריכים להשתנות ומסלול החישוב שלהם לא השתנה ולכן לא נשנה דבר.
- לערכי $partialNen$ של B צריכים להתווסף כל ערכי ה- n של A מכיוון שכעת הם הצטרפו לקבוצה כרונולוגית אחרי חברי קבוצת A .
לכן נוסיף לערך ה- $relNen$ של שורש B את $squadNen$ של הקבוצה A . כלומר:
 $Broot.relNen += A.squadNen$
- ערכי $partialNen$ ו- $fight$ של חברי הקבוצה A לא צריכים להשתנות אך כעת מסלול החישוב שלהם כולל גם את שורש העץ B . לכן נתקן זאת ע"י הורדת $relNen$ ו- $relFights$ של שורש B מהערכים הנ"ל של שורש A בהתאמה. כלומר:
 $A.relNen = B.relNen$
 $A.relFights = B.relFights$
- נעדכן את ערכי רשומת A להכיל את מידע הקבוצה המאוחדת.
- ננתק את שורש A מהרשומה ונחבר את הרשומה לשורש B .
- נוציא ונמחק את רשומת B מעץ הרשומות.

סיבוכיות הזמן:

תחילה אנו מבצעים שתי חיפושים בעץ הרשומות. פעולות אלו מתבצעות ב- $O(\log k)$. לאחר מכן, אנו מבצעים סדרה קבועה של פעולות השמה וחישוב ולכן פעולות אלו מתבצעות ב- $O(1)$. כלומר בסך הכל סיבוכיות הזמן היא:

$$O(\log k) + O(1) = O(\log k)$$

כיווץ מסלולים $compress(node)$:

פונקציית עזר המבצעת כיווץ רקורסיבית. בהינתן מצביע ל $UnionNode$. עוברת על הצמתים עד אשר מגיעה לשורש. לאחר מכן בקיפול אחורה תולה מחדש את כל הצמתים עליהם עברה על השורש. בכל תליה של צומת, הפונקציה מוסיפה לערכי של $relFights$ ו- $relNen$ של הצומת את הערכים הנ"ל של ההורה שלה, על מנת לשמור על נכונותם. כלומר, מסלול החיפוש החדש עובר רק על הצומת והשורש ולכן נתקן אותו בכך שבצומת נשמור את הסכום על מסלול הסכימה הישן לא כולל השורש.

נבחין כי במבנה שלנו, לא קיימת פעולת $find$ כמו במימוש הסטנדרטי של מבנה $Union$. לכן כתחליף, שיפור כיווץ המסלולים יתבצע כאשר נקרא לפונקציות $get_hunter_fights_number$ ו- $get_partial_nen_ability$.

הסבר הפונקציות:

הערה: במבנה $Huntech$ אנו עושים שימוש במבנה $Union$ המכיל בתוכו טבלת ערבול דינמית ועץ AVL . לצורך פשטות ההסבר והאלגוריתם, אנו נתאר את הפעולות המתבצעות גם בתוך מבנה ה- $Union$ (כגון הכנסה לעץ הרשומות ולטבלת הערבול המוחזקים ע"י מבנה ה- $Union$) כחלק ישיר מתיאור הפונקציה, גם אם בפועל הדבר מתבצע ע"י קריאה למתודה של $Union$.

$Huntech()$:

במבנה הנתונים אנחנו מחזיקים את $AuraTree$ ואת $squads$

$AuraTree$ - מאתחל עץ ריק עם מצביע $nullptr$ וגודל 0.
: $squads$

- $hunterHash$ - מאתחל מערך מצביעים ריקים בגודל 8, ומתאחל 2 שדות בערכים קבועים
- $groups$ - מאתחל עץ ריק עם מצביע $nullptr$ וגודל 0.

באתחול אין קבוצות ולכן אתחול העצים מתבצע ב- $O(1)$. עבור טבלת הערבול, אנו מתחילים בטבלה בגודל קבוע. כלומר אתחול מערך ריק בגודל קבוע ולכן $O(1)$. לכן כל פעולת האתחול תתבצע ב- $O(1)$.

$\sim huntech()$:

תחילה נבחין כי מחיקת כל העצמים הפשוטים ($Hunter, Node$ וכו') מתבצע ב- $O(1)$ מכיוון שכל עצם כזה מחזיק כמות קבועה של שדות הנמחקים ב- $O(1)$. כעת נמנה את מספר העצמים שעלינו לשחרר. בכל רגע נתון אנחנו מחזיקים פעמיים את מספר הקבוצות בשני עצי ה- AVL לכן $2k$. גודל טבלת הערבול היא $2n$ לכל היותר, ויש בה בדיוק n $HashNodes$. לכן מעבר עליה ומחיקת האיברים בה ייקח $O(3n)$. עבור 2 עצי ה- AVL נעבור בסיור $preorder$ ונמחק את הצמתים ולכן $O(2k)$.

לכן סיבוכיות הזמן היא:

$$O(3n) + O(2k) = O(n + k)$$

:add_squad(int squadId)

ראשית נבצע בדיקת קלט, במידה והקלט לא תקין נחזיר *INVALID_INPUT*.
נסה להכניס את הקבוצה אל עץ הרשומות, במידה והקבוצה כבר קיימת נחזיר *FAILURE*.
במקרה של חריגת זיכרון בהכנסה הנ"ל נחזיר *ALLOCATION_ERROR*.
(נבחין כי אם התקבלה חריגת זיכרון ויצאנו לא שינינו את המבנה).
אחרת, ניצור זוג (*groupId, Aura = 0*) ונכניס אותו ל- *AuraTree*.
אם בשלב זה נזרקה חריגת זיכרון, נוציא מעץ הרשומות את הקבוצה שיצרנו ונחזיר *ALLOCATION_ERROR*.

לבסוף, נחזיר *SUCCESS*.

נבחין כי לפי אופן המימוש, במידה והקבוצה כבר קיימת התהליך ייכשל, כך שלא צריך לבצע חיפוש לפני.

ביצענו שתי הכנסות לעצי *AVL* המכילים k איברים ולכן סיבוכיות הזמן היא:

$$2 \cdot O(\log k) = O(\log k)$$

:remove_squad(int squadId)

ראשית נבצע בדיקת קלט, במידה והקלט לא תקין נחזיר *INVALID_INPUT*.
נבצע חיפוש בעץ *squadIdTree* כדי לבדוק האם הקבוצה קיימת במערכת.
במידה והקבוצה לא קיימת, נחזיר *FAILURE*.
במידה וקיימת, נסירה מן העצים *squds* ו- *AuraTree*.
נבחין כי לפי הגדרה, לא נוציא את חברי הקבוצה מטבלת הערבול ומבנה ה- *Union*.
לבסוף, נחזיר *SUCCESS*.

ביצענו חיפוש ושתי הוצאות מעצי *AVL* המכילים k איברים ולכן סיבוכיות הזמן היא:

$$3 \cdot O(\log k) = O(\log k)$$

:add_hunter(int hunterId,...)

ראשית, נבדוק את תקינות הפרמטרים, במידה ואחד מהם לא תקין, נחזיר *INVALID_INPUT*.
נבצע חיפוש של הקבוצה בעץ *groups*.
במידה והקבוצה לא קיימת, נחזיר *FAILURE*.
במידה וה- *Aura* של הצייד גדולה מ-0 כלומר ה- *Aura* הכוללת של הקבוצה עם הצייד החדש שונה מזו של הקבוצה בלעדיו, נכניס לעץ *auraTree* זוג חדש של מזהה הקבוצה עם ההילה העדכנית.
נבחין כי ההילה החדשה בהכרח שונה מהקודמת ולכן לא תיווצר התנגשות בעץ.
אם התקבלה חריגת זיכרון נסיים ונחזיר *ALLOCATION_ERROR*. נבחין כי לא שינינו את המבנה.
אחרת, נכניס את הצייד החדש אל הקבוצה בעזרת הפונקציה *insertHunterToGroup* שתוארה קודם.
אם ההכנסה נכשלה עקב כך שהצייד כבר היה קיים במערכת או מחריגת זיכרון, נוציא מ- *auraTree* את הזוג החדש שהכנסנו ונחזיר *FAILURE* או *ALLOCATION_ERROR* בהתאמה. נבחין כי גם כאן, מבנה הנתונים נשאר ללא שינוי לאחר השגיאות.
(פונקציית *insertHunterToGroup* מחזירה 0 במידה ומנסים להכניס צייד שכבר קיים כפי שתואר קודם).

לבסוף, במידת הצורך (אם הכנסנו זוג חדש לעץ *auraTree*) נוציא את הזוג המייצג את הקבוצה לפני ההכנסה מהעץ *auraTree*.

לבסוף נחזיר *SUCCESS*.

אנו מבצעים פעולות חיפוש, מחיקה והכנסה אל עץ AVL ובנוסף שימוש בפונקציה *insertHunterToGroup* ולכן הסיבוכיות היא:

$$O(\text{insertHunterToGroup}) + 3 \cdot (\log k)$$

הסיבוכיות המשוערכת של *insertHunterToGroup* היא $O(\log k)$ ולכן הסיבוכיות המשוערכת של הפונקציה היא:

$$O(\log k)$$

:output t < int > squad_duel(int squadId1, int squadId2)

ראשית, נבדוק את תקינות הקלטים, במידה ואחד מהם לא תקין, נחזיר *INVALID_INPUT*.
נבצע חיפוש אחר שתי הקבוצות, אם אחת לא קיימת, או אם אחת מהם ריקה, נחזיר *FAILURE*.
נוסיף 1 למשתנה *relFights* של השורש של כל אחד מהקבוצות.
נבדוק מי המנצח ואת סיבת הניצחון (מספר קבוע של פעולות).
בהתאם לתוצאות הקרב, נגדיל את *squadExp* של הקבוצות.
בהתאם לתוצאות הקרב, נחזיר *SUCCESS* עם הערך המתאים.
סך הכל ביצענו מספר קבוע של פעולות $O(1)$ וחיפוש עבור הקבוצות בעץ הקבוצות, ולכן סיבוכיות הזמן היא:

$$c \cdot O(1) + O(\log k) = O(\log k)$$

:get_hunter_fights_number(int hunterId)

ראשית, נוודא את תקינות הקלט, במידה והוא לא תקין, נחזיר *INVALID_INPUT*.
נחפש את הצייד בטבלת הערבוד.
במידה ולא מצאנו אותו, נחזיר *FAILURE*.
אחרת, נבצע כיווץ מסלולים ולאחר מכן נחשב לפי הנוסחה את כמות הקרבות שבהם הצייד השתתף:

$$\text{fights} = (\text{hunter.fightsHad} - \text{hunter.joinedGroupFights}) + \sum \text{Node.relFights}$$

למעשה, הסכימה מתבצעת רק על הצומת עצמו והשורש.
לבסוף, נחזיר *SUCCESS*.

נבחין כי אנו משתמשים בשיפורי כיווץ מסלולים ואיחוד לפי גודל.
כפי שהוסבר בפעולת *compress()*, פעולות *get_hunter_fights_number* ו-
get_partial_nen_ability מהוות תחליף לפעולת ה- *find* במבנה *Union* רגיל.

ולכן מהרצאה, סיבוכיות הזמן המשוערכת בממוצע על הקלט של פעולות *Unite, Find* הינה $O(\log^* n)$.
פעולת הסכימה על המסלול לשורש זהה לפעולת *Find* (פרט לסכימת שדה בכל צומת ב- $O(1)$).
לכן סיבוכיות הזמן שלה גם היא $O(\log^* n)$.

:get ith collective aura squad(int i)

ראשית, נוודא את תקינות הקלט.
אם גודל האיברים בעץ הוא 0 או i גדול מכמות הצוותים בעץ *Aura* או קטן ממש 1, נחזיר *FAILURE*.
אחרת, נבצע את החיפוש ב- *AuraTree* לפי אלגוריתם החיפוש של צומת בעל אינקס i בעץ דרגות הנלמד בתרגול.
ראינו כי סיבוכיות האלגוריתם היא $\log k$.

:get partial nen ability(int hunterId)

ראשית, נוודא את תקינות הקלט, במידה והוא לא תקין, נחזיר *INVALID_INPUT*.
אופן פעולת הפונקציה זהה לזה *get_hunter_fight_number* למעט שימוש בנוסחה הבא:

$$partial_nen = hunter.NenAbility + \sum Node.relNen$$

ולכן באותו אופן, סיבוכיות הפונקציה היא $O(\log^* n)$

:force join(int forcingSquadId,int forcedSquadId)

ראשית, נוודא בדיקה אם הקלט תקין, במידה ולא, נחזיר *INVALID_INPUT*.
נבצע חיפוש ב- *Union* אחר שני הקבוצות, במידה ואחת לא קיימת, או שהקבוצה הכופה גודלה 0, נחזיר *FAILURE*.
כעת נבדוק אם תנאי הכפייה מתקיים, אם לא נחזיר *FAILURE*.
אחרת נבצע איחוד לפי גודל הקבוצות כפי שהוסבר קודם.
נסיר את הקבוצה הנכפת מעץ ה-*squads*, ונסיר את שני הקבוצות מעץ ה-*auran*.
נכניס את הקבוצה החדשה מחדש אל עץ ה-*auran*.
סך הכל אנחנו מבצעים 2 פעולות חיפוש, פעולת איחוד, 3 פעולות הוצאה, ופעולת הכנסה.
ולכן הסיבוכיות היא:
אנחנו מבצעים כאן פעולת איחוד, חיפוש במבנה *union*, 4 פעולות חיפוש בעץ *AVL*, 2 הסרות, והכנסה.
ושינוי קבוע של שדות.
לכן הסיבוכיות היא:

$$c \cdot O(1) + (\log^* n) + 6 \cdot (\log k) = O(\log^* n + \log k)$$

סיבוכיות מקום:

גודל של עצמים *squad*, *hunter* הוא $O(1)$, כיוון שהן מכילים מספר קבוע של שדות.
בכל רגע נתון אנחנו שומרים 2 עצמי *AVL* עם k צמתים.
מערך $HunterHash = O(2 \cdot n)$.
ומבנה *union/find* שמכיל n צמתים.
ולכן סיבוכיות המקום היא:

$$2 \cdot k \cdot O(1) + O(2 \cdot n) + n = O(n + k)$$